



ASIGNATURA: Sistemas Robóticos Móviles

Actividad 2: Programación y control de un robot terrestre tipo Pioneer en Coppeliasim/Lua

1. Organización

1.1. Datos de la actividad

La presente actividad corresponde a la Actividad 2 de la asignatura de Sistemas Robóticos Móviles. En esta tarea, se deberá realizar la programación de un robot tipo Pioneer en el CoppeliaSim/Lua. Su ponderación representa el 30% de la calificación total del portafolio de la asignatura. El portafolio, a su vez, consta de dos actividades evaluadas, cada una con un peso del 30% sobre la nota final de la asignatura. A estas ponderaciones se sumará la calificación de la prueba final, que equivale al 40% del total.

La entrega de esta actividad está programada para el martes 26 de mayo de 2026 antes de las 23:59 en la primera convocatoria. No realizar la entrega antes supondrá una calificación de 0 en el 30% correspondiente.

La actividad deberá ser enviada a través de la plataforma del campus virtual.

Para cualquier consulta, podéis escribir al siguiente correo electrónico:

jiniguez@professor.universidadviu.com

2. Introducción

En esta actividad, vamos a programar un robot móvil tipo Pioneer P3DX en el simulador CoppeliaSim. La actividad se divide en dos actividades principales: La primera de ellas se compone de la programación del robot para que se desplace a un punto establecido; mientras que la segunda de ellas se compone de la programación de un sistema de evitación de colisiones.

En particular, la primera parte de la actividad replica un ejemplo estudiado en clase. En dicho ejemplo se controla un robot mediante el método de campos potenciales para navegar de forma reactiva y directa hacia un objetivo específico.

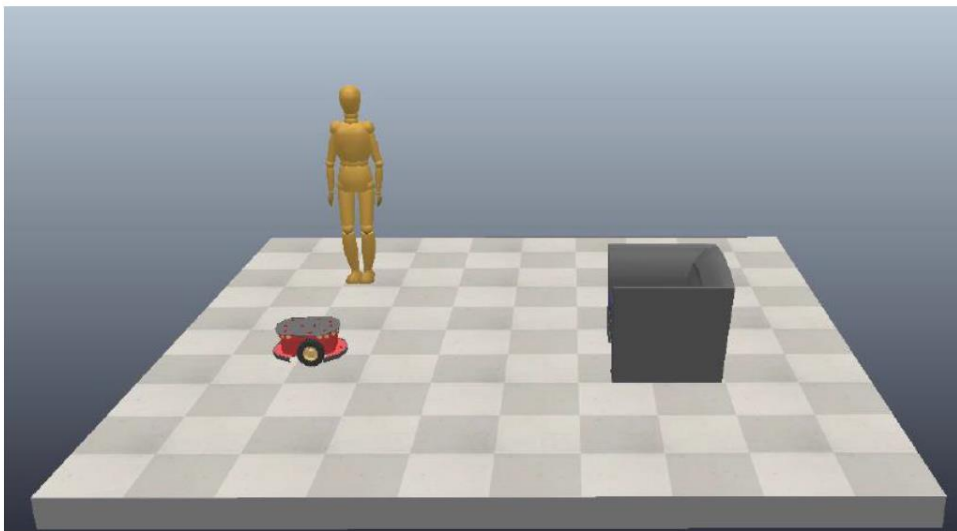
La segunda parte de la actividad construye sobre un ejemplo de código proporcionado en la asignatura un programa para evitar obstáculos a la vez que se emplea el método realizado en la primera parte.

Parte 1: Desplazamiento del Robot Pioneer P3DX

Construye en CoppeliaSim un escenario compuesto por:

- Robot/mobile/PioneerP3DX
- People/mannequin
- Furniture/chairs/sofa

Tu escenario debería verse de la siguiente manera:



Programa el Pioneer para “sentir” una fuerza de atracción hacia el objeto “mannequin” en la simulación. Para poder realizar esto, puedes basarte en el siguiente código estudiado en clase:

```
1 -----
2 -- Cargamos la librería principal de CoppeliaSim
3 -----
4 sim = require('sim')
5
6 -----
7 -- Se ejecuta UNA VEZ al empezar la simulación
8 -----
9 function sysCall_init()
10
11     -- Obtenemos el robot Pioneer
12     pioneer = sim.getObject("/Pioneer3DX")
13
14     -- Obtenemos el objeto Bill (objetivo)
15     bill = sim.getObject("/Bill")
16
17     -- Variables para guardar la pose del robot y el objetivo
18     Robotpose = {0,0,0,0,0,0}
19     goal = {0,0}
20
21     -- Obtenemos los motores del robot
22     motorLeft = sim.getObject("/leftMotor")
23     motorRight = sim.getObject("/rightMotor")
24 end
25
26 -----
27 -- Función que devuelve la posición y orientación del robot
28 -----
29 function get_RobotPose()
30
31     -- Posición del robot: (x,y,z)
32     local pos = sim.getObjectPosition(pioneer, -1)
33
34     -- Orientación del robot: (rotX, rotY, rotZ)
35     local ori = sim.getObjectOrientation(pioneer, -1)
36
37     -- Imprimimos SOLO la orientación en Z (yaw) en grados
38     print("orientacion=" .. string.format("%.2f", math.deg(ori[3])))
```

```

40     -- Devolvemos todo junto en una tabla
41     return {pos[1], pos[2], pos[3], ori[1], ori[2], ori[3]}
42 end
43
44 -----
45 -- Funci?n que devuelve la posici?n de Bill
46 -----
47 function get_BillPose()
48
49     -- Posici?n de Bill: (x,y,z)
50     local pos = sim.getObjectPosition(bill, -1)
51
52     -- Solo usamos x e y
53     return {pos[1], pos[2]}
54 end
55
56 -----
57 -- Se ejecuta en cada paso de simulaci?n (sensores)
58 -----
59 function sysCall_sensing()
60
61     -- Leemos la pose del robot
62     Robotpose = get_RobotPose()
63
64     -- Leemos la posici?n del objetivo
65     goal = get_BillPose()
66
67     -- Imprimimos
68     print(Robotpose)
69
70     print(goal)
71 end
72
73 -----
74 -- Calcula las velocidades para ir hacia Bill
75 -----
76 function desplazamiento(Robotpose, goal)
77
78     -- Fuerza de atracci?n (qu? r?pido va hacia Bill)
79     local f_atracc = 0.5
80
81     -- Vector desde el robot hasta Bill
82     local dx = goal[1] - Robotpose[1]
83     local dy = goal[2] - Robotpose[2]
84
85     -- ?ngulo hacia Bill
86     local ang_goal = math.atan2(dy, dx)
87

```

```

94     -- Control simple tipo Braitenberg
95
96     -- Caso 1: Robot bien orientado          Caso 2: Hay que girar a la izquierda          Caso 3: Hay que girar a la derecha
97     -----error_ang = 0                      error_ang > 0                      error_ang < 0
98     -----vLeft = f_atracc*dist              vLeft = f_atracc*dist - error          vLeft = f_atracc*dist + error
99     -----vRight = f_atracc*dist             vRight = f_atracc*dist + error         vRight = f_atracc*dist - error
100    -----Ambas ruedas iguales                Rueda izquierda m?s lenta             Rueda izquierda m?s r?pida
101    -----Robot avanza recto                  Rueda derecha m?s r?pida              Rueda derecha m?s lenta
102                                           --Robot gira a la izquierda              Robot gira a la derecha
103
104
105     local vLeft = f_atracc * dist - error_ang
106     local vRight = f_atracc * dist + error_ang
107
108     return vLeft, vRight
109 end
110
111 -----
112 -- Se ejecuta en cada paso (actuadores)
113 -----
114 function sysCall_actuation()
115
116     -- Calculamos velocidades
117     local vLeft, vRight = desplazamiento(Robotpose, goal)
118
119     -- Enviamos las velocidades a los motores
120     sim.setJointTargetVelocity(motorLeft, vLeft)
121     sim.setJointTargetVelocity(motorRight, vRight)
122 end

```

Añade el robot Pioneer con el script para que siga a Bill a una celda robotizada, de manera que se integre en la aplicación robótica específica y trabaje de forma coordinada con los otros sistemas que componen la celda. Puedes utilizar la celda robotizada creada en otras asignaturas.

Parte 2: Programación de un sistema anticollisiones

Modifica el código anterior para que el robot avance hacia el objetivo y esquivе los objetos que se encuentre en su camino. Para poder realizar esto, puedes basarte en el siguiente código estudiado en clase:

```
Simulation script "/Pioneer3DX/Script"
LUA
1  -----
2  -- Cargamos la librería principal de CoppeliaSim
3  -----
4  sim = require('sim')
5  -----
6  -- Se ejecuta UNA VEZ al empezar la simulación
7  -----
8  function sysCall_init()
9
10     -- Obtenemos el robot Pioneer
11     pioneer = sim.getObject("/Pioneer3DX")
12
13     -- Obtenemos el objeto Bill (objetivo)
14     bill = sim.getObject("/Bill")
15
16     -- Variables para guardar la pose del robot y el objetivo
17     Robotpose = {0,0,0,0,0,0}
18     goal = {0,0}
19
20     -- Obtenemos los motores del robot
21     motorLeft = sim.getObject("/leftMotor")
22     motorRight = sim.getObject("/rightMotor")
23
24     -----
25     -- Sensores ultrasónicos
26     -----
27     usensors = {}
28     for i=1,16 do
29         usensors[i] = sim.getObject("/Pioneer3DX/ultrasonicSensor",{index=i-1})
30     end
31
32     -----
33     -- Parametros
34     -----
35     noDetectionDist = 0.5 -- Distancia mínima a la que el sensor empieza a influir
36     maxDetectionDist = 0.2 -- Distancia máxima para evitar divisiones
37
38     detect = {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0} -- Tabla donde guardamos cuánto detecta cada sensor (0 a 1)
39
40     braitenbergL = {-0.2,-0.4,-0.6,-0.8,-1,-1.2,-1.4,-1.6,0,0,0,0,0,0,0,0} -- Pesos para la rueda izquierda
41
42     braitenbergR = {-1.6,-1.4,-1.2,-1,-0.8,-0.6,-0.4,-0.2,0,0,0,0,0,0,0,0} -- Pesos para la rueda derecha
43
44 end
```

```

44
45 -----
46 -- Funcion que devuelve la pose del robot
47 -----
48 function get_RobotPose()
49     local pos = sim.getObjectPosition(pioneer, -1)
50     local ori = sim.getObjectOrientation(pioneer, -1)
51     return {pos[1], pos[2], pos[3], ori[1], ori[2], ori[3]}
52 end
53
54 -----
55 -- Funcion que devuelve la posici?n de Bill
56 -----
57 function get_BillPose()
58     local pos = sim.getObjectPosition(bill, -1)
59     return {pos[1], pos[2]}
60 end
61
62 -----
63 -- Sensores
64 -----
65 function sysCall_sensing()
66     Robotpose = get_RobotPose()
67     goal = get_BillPose()
68 end
69
70 -----
71 -- Seguimiento de Bill
72 -----
73 function desplazamiento(Robotpose, goal)
74
75     local dx = goal[1] - Robotpose[1]
76     local dy = goal[2] - Robotpose[2]
77
78     local ang_goal = math.atan2(dy, dx)
79     local error_ang = ang_goal - Robotpose[6]
80
81     -- Normalizamos el error angular para que est? entre -pi y pi
82     -- Esto evita giros absurdos de m?s de 360 grados
83
84     while error_ang > math.pi do error_ang = error_ang - 2*math.pi end
85     while error_ang < -math.pi do error_ang = error_ang + 2*math.pi end
86

```

```

87 -- Distancia al objetivo
88 local dist = math.sqrt(dx*dx + dy*dy)
89 local f_atracc = 0.5
90
91 -----
92 -- Control simple tipo Braitenberg
93
94 -- Caso 1: Robot bien orientado      Caso 2: Hay que girar a la izquierda      Caso 3: Hay que girar a la derecha
95 -----error_ang = 0      error_ang > 0      error_ang < 0
96 -----vLeft = f_atracc*dist      vLeft = f_atracc*dist - error      vLeft = f_atracc*dist + error
97 -----vRight = f_atracc*dist      vRight = f_atracc*dist + error      vRight = f_atracc*dist - error
98 -----Ambas ruedas iguales      Rueda izquierda m?s lenta      Rueda izquierda m?s r?pida
99 -----Robot avanza recto      Rueda derecha m?s r?pida      Rueda derecha m?s lenta
100 -----Robot gira a la izquierda      Robot gira a la derecha
101
102 local vLeft = f_atracc * dist - error_ang
103 local vRight = f_atracc * dist + error_ang
104
105 return vLeft, vRight
106 end
107
108 -----
109 -- Actuadores
110 -----
111 function sysCall_actuation()
112
113     -- Velocidades base hacia Bill
114     local vLeft, vRight = desplazamiento(Robotpose, goal)
115
116     -- Lectura de sensores
117     -----
118     for i=1,16 do
119         local res, dist = sim.readProximitySensor(usensors[i]) -- res = 1 si detecta un objeto, 0 si no. dist = distancia al objeto detectado
120
121         if res > 0 and dist < noDetectionDist then
122             if dist < maxDetectionDist then
123                 dist = maxDetectionDist
124                 -- Si el objeto est? demasiado cerca, limitamos la distancia m?nima para evitar errores
125             end
126         end
127     end
128

```

```

127         -- Convertimos la distancia en un valor entre 0 y 1
128         -- 1 = obst?culo muy cercano
129         -- 0 = obst?culo lejano
130
131         detect[i] = 1 - ((dist - maxDetectionDist) / (noDetectionDist - maxDetectionDist))
132     else
133         detect[i] = 0
134     end
135 end
136
137 -----
138 -- Correcci?n por evitaci?n
139 -----
140 for i=1,16 do
141
142     -- Modificamos la velocidad de la rueda izquierda y derecha
143     -- seg?n cu?nto detecta el sensor i
144     -- braitenbergL[i] indica cu?nto influye ese sensor
145
146     vLeft = vLeft + braitenbergL[i]*detect[i]
147     vRight = vRight + braitenbergR[i]*detect[i]
148 end
149
150 -- Enviamos velocidades a los motores
151 sim.setJointTargetVelocity(motorLeft, vLeft)
152 sim.setJointTargetVelocity(motorRight, vRight)
153 end

```

Añade el robot Pioneer con el script para que siga a Bill a una celda robotizada, de manera que se integre en la aplicación robótica específica y trabaje de forma coordinada con los otros sistemas que componen la celda. Puedes utilizar la celda robotizada creada en otras asignaturas.

Tareas a completar

- Integración del robot móvil en la celda robotizada en CoppeliaSim.
- Documento en PDF
- Presentación en clase con Powerpoint.

Si el script del robot móvil es exactamente igual al visto en clase y entregado a los alumnos la nota será aprobado. A medida que el estudiante amplie el script y por lo tanto las funciones del robot móvil en la celda robotizada, la nota será más alta, de manera proporcional a las mejoras añadidas.